ESCOLA POLITÉCNICA DA UNIVERSIDADE DE SÃO PAULO
Departamento de Engenharia de Computação e Sistemas Digitais (PCS)
PCS 3858 — Laboratório de Sistemas Embarcados — Turma 3
Prof. Dr. Carlos Eduardo Cugnasca

Auditor Visual de Medicação

Relatório Técnico — Prova P1
Progresso do Projeto Dirigido até 16/10/2026

Gabriel Dias Tambelli NUSP 12551370
Henrique Murakami de Paula NUSP 12551154
Juliana Marcarini Carloni NUSP 12553538

Repositório: github.com/Tambelli/auditor-de-medicacao (ramo `mvp-autonomo-telegram`)
São Paulo, outubro de 2026

Sumário

1. Introdução e estado atual
2. Evolução do escopo
3. Especificação de requisitos
4. Arquitetura (produto, hardware e software)
5. Projeto do hardware
6. Projeto do software
7. Protocolos de comunicação
8. Ferramentas, linguagens e sistema operacional
9. Interface com o usuário
10. Protótipo
11. Verificação e resultados obtidos
12. Problemas enfrentados
13. Cronograma previsto e realizado
14. Divisão de tarefas
15. Próximos passos até a P2
16. Referências

1. Introdução e estado atual

A não adesão ao tratamento medicamentoso é frequente entre idosos e portadores de doenças crônicas, e o esquecimento de doses gera complicações e sobrecarga para familiares e cuidadores. O **Auditor Visual de Medicação** é um dispositivo passivo, colocado sobre um porta-comprimidos convencional, que verifica com uma câmera se a dose do dia foi retirada do compartimento e avisa o cuidador quando isso não acontece.

O protótipo não dispensa comprimidos, não identifica o medicamento e não comprova a ingestão: ele observa o estado do compartimento (dose presente ou retirada) dentro de uma janela de horário. Todo o processamento de imagem e a decisão ocorrem no próprio microcontrolador.

> **Estado na data da P1.** O MVP autônomo está implementado e funcionando na placa: o ESP32-CAM opera ligado apenas a uma fonte, obtém a hora pela Internet (SNTP), executa a verificação diária no horário programado, classifica o compartimento com visão computacional embarcada e avisa o cuidador pelo **Telegram**, com foto quando é preciso conferir. A configuração (Wi-Fi, Telegram, horário, região de interesse e calibração) é feita por um painel no computador, via USB. Wi-Fi, SNTP, câmera, calibração e o envio de mensagens e fotos ao Telegram foram verificados na placa. Faltam os ensaios quantitativos de acurácia e de estabilidade (seção 15).

2. Evolução do escopo

O projeto seguiu a **espiral evolucionária** prevista na metodologia: a cada ciclo, os requisitos foram revistos à luz do que foi aprendido em bancada. A tabela resume as três versões do escopo.

Aspecto	Proposta inicial (set/2026)	Plano consolidado (Tarefa 02, 18/09)	MVP atual (out/2026)
Placa	Seeed XIAO ESP32-S3 Sense	Waveshare ESP32-S3-ETH + câmera OV2640-V3 avulsa	**ESP32-CAM RoboCore com gravador integrado** (ESP32 clássico, OV2640 integrada)
Classificação	Rede neural convolucional (TinyML)	Comparação com referências cheio/vazio	**Só a referência "vazio"**: qualquer objeto no compartimento é dose (seção 6.3)
Hora	RTC DS3231 externo	Ajuste manual pela serial	**SNTP pela Internet** (ajuste manual mantido para bancada)
Alerta	Wi-Fi ao cuidador	Buzzer + botão; Wi-Fi opcional	**Telegram com foto**; sem buzzer e sem botão

Alimentação	Bateria 18650 + deep sleep	Fonte 5 V; PC para configurar e registrar	Fonte USB 5 V; **funciona sem computador**
Compartimentos	7 dias × 3 horários	1 compartimento	1 compartimento, 1 dose por dia

Por que mudou. A CNN exigiria um conjunto de imagens rotuladas que o grupo não possui. A comparação com uma referência calibrada na própria cena é explicável, não precisa de treinamento e cabe na memória do ESP32 clássico. A ESP32-CAM já traz câmera, conector e gravador USB, o que eliminou o risco de compatibilidade elétrica e mecânica da câmera avulsa prevista para a Waveshare. Na revisão de outubro, o grupo decidiu que o produto precisa funcionar só ligado a uma fonte, avisando o cuidador à distância. Um buzzer avisa apenas quem está perto do aparelho, então o canal remoto (Telegram) passou de extensão opcional a requisito principal, e buzzer e botão saíram do escopo. O organizador semanal de 7 compartimentos ficou como extensão.

3. Especificação de requisitos

Os requisitos partem da Tarefa 02 e foram revisados para o MVP autônomo. Situação: VERIFICADO na placa; IMPLEMENTADO e testado em software, falta ensaio na placa; PENDENTE. A etiqueta ALTERADO indica requisito revisto na mudança de escopo.

3.1 Requisitos funcionais

ID	Requisito	Situação
RF01	Configurar horário da dose, tolerância, intervalo de captura, Wi-Fi e Telegram pelo computador (USB); os valores ficam na placa.	VERIFICADO
RF02	Calibrar e salvar a região de interesse (ROI) e a referência do compartimento vazio, com medição do ruído; conferir opcionalmente se a dose se destaca.	ALTERADO VERIFICADO
RF03	Executar a verificação automaticamente todo dia no horário programado, sem o computador.	ALTERADO IMPLEMENTADO
RF04	Classificar cada leitura como dose presente, vazio ou inconclusiva, para **qualquer comprimido em qualquer posição** do compartimento.	ALTERADO VERIFICADO (4 situações; ensaio RNF03 pendente)
RF05	Confirmar retirada só se a dose foi vista na janela e depois houve leituras vazias consecutivas; vazio sem dose vista vira “não reposta”.	IMPLEMENTADO
RF06	Avisar o cuidador se a dose continuar no compartimento ao fim da tolerância, com foto, e continuar observando para avisar uma retirada atrasada.	ALTERADO (era buzzer) IMPLEMENTADO
RF07	Informar o estado do sistema pela serial (painel) e pelo Telegram.	ALTERADO (sem botão) VERIFICADO
RF08	Registrar capturas, classificações, transições e falhas em JSON pela serial; enviar o resultado diário ao Telegram.	VERIFICADO
RF09	Detectar falha de câmera, imagem inválida e falta de hora; avisar e nunca declarar retirada nesses casos.	IMPLEMENTADO

RF10	Manter configuração, calibração e credenciais em memória não volátil; após reinício, recuperar a hora pelo SNTP e rearmar sozinho, sem inferir retirada no período desligado.	ALTERADO VERIFICADO (persistência e aviso de boot)
RF11	Rearmar automaticamente a verificação do dia seguinte, sem reutilizar o resultado anterior.	ALTERADO IMPLEMENTADO
RF12	Notificar o cuidador pelo Telegram via Wi-Fi; sem rede, a verificação continua e as mensagens aguardam na fila.	ALTERADO (era opcional) VERIFICADO

3.2 Requisitos não funcionais

ID	Requisito	Situação
RNF01	Execução embarcada: aquisição, classificação, agenda e decisão no ESP32; o computador serve só para configurar.	VERIFICADO
RNF02	Desempenho: até 2 s por leitura (2 quadros + análise); aviso enviado logo após o fim da janela.	VERIFICADO leitura: 168–220 ms
RNF03	Qualidade: ≥ 90% de acerto por classe em 30 observações de cada classe fora da calibração; zero retiradas indevidas em 10 oclusões.	PENDENTE (ensaio previsto)
RNF04	Reprodutibilidade: suporte, distância, iluminação e objetos documentados.	EM ANDAMENTO (suporte fixo em construção)
RNF05	A decisão independe da rede; só o envio do aviso depende do Wi-Fi.	ALTERADO IMPLEMENTADO
RNF06	Estabilidade: 20 ciclos consecutivos sem reinício; watchdog de tarefas; memória acompanhada no status.	PENDENTE (watchdog implementado)
RNF07	Integração elétrica: fonte de 5 V / 2 A sem reinícios por queda de tensão (brownout).	EM ANDAMENTO
RNF08	Usabilidade: configuração guiada pelo painel; mensagens claras ao cuidador.	VERIFICADO
RNF09	Dados: imagens processadas localmente; foto pequena (160 × 120) enviada apenas ao chat do cuidador, só quando é preciso conferir; senhas e token nunca aparecem nos registros.	ALTERADO VERIFICADO
RNF10	Manutenibilidade: módulos separados, testes automatizados e integração contínua.	VERIFICADO
RNF11	Custo: usar a placa disponível, sem compras eletrônicas adicionais.	VERIFICADO
RNF12	Portabilidade das ferramentas: painel, compilação e testes no Windows, macOS e Linux.	VERIFICADO (CI no Ubuntu e no macOS)

4. Arquitetura

4.1 Arquitetura do produto (uso)

O ciclo de uso tem duas fases. Na **instalação**, feita uma única vez com o computador, o cuidador posiciona a câmera sobre o compartimento, informa Wi-Fi e Telegram, escolhe o horário, recorta a região do compartimento e calibra as imagens “vazio” e “presente”. Na **operação**, o aparelho fica ligado só na fonte.

O cuidador repõe a dose a qualquer momento antes do horário; na janela, o aparelho verifica a retirada e envia o resultado ao celular.

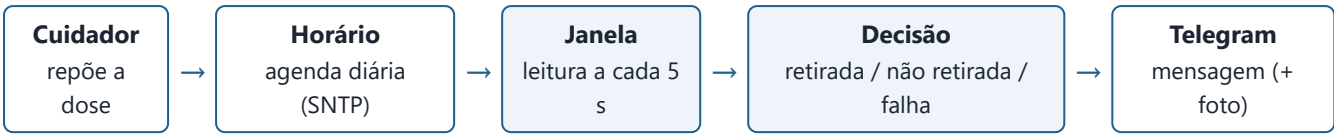


Figura 1 — Fluxo de uso diário.

4.2 Arquitetura de hardware

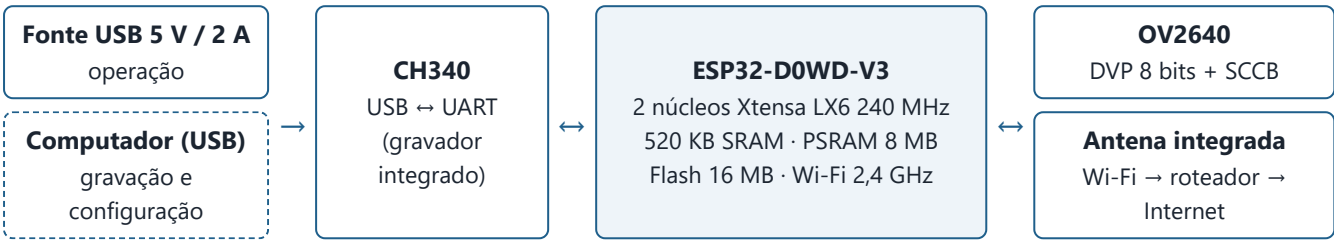


Figura 2 — Diagrama de blocos do hardware. Não há componentes eletrônicos externos.

4.3 Arquitetura de software

O firmware é escrito em C++17 sobre ESP-IDF 5.4.1 e FreeRTOS. O **núcleo** (`components/auditor`) contém a visão, a classificação, a agenda e a máquina de estados, sem depender de hardware. Por isso ele compila e é testado no computador. Os módulos de plataforma ficam em `main/`.

Módulo	Responsabilidade
`components/auditor`	Extração de características (ROI 32×32 e entorno), distâncias, classificação, cálculo da próxima janela e máquina de estados `Cycle`.
`main/app_main.cpp`	Laço principal: comandos USB, supervisão (Wi-Fi, hora, câmera), agenda, leituras e transições; monta as mensagens ao cuidador.
`main/camera.cpp`	Único dono do driver `esp32-camera`; captura JPEG, converte para cinza e guarda o último JPEG para a foto.
`main/network.cpp`	Wi-Fi em modo estação com reconexão por espera crescente; SNTP com dois servidores; fuso UTC−3.
`main/notify.cpp`	Tarefa do Telegram com fila de 8 mensagens, HTTPS e novas tentativas.
`main/storage.cpp`	Configuração e referências versionadas em NVS (memória não volátil).
`main/events.cpp`	Eventos JSON na serial, com tempo monotônico e hora civil.
`tools/` (Python)	Painel gráfico, console, protocolo, avaliador de ensaios, compilação multiplataforma e testes.

Tarefas FreeRTOS

Tarefa	Prioridade / pilha	Função
`main` (app_main)	padrão · 24 KB	Laço de 100 ms: comandos, supervisão, agenda e leituras. Monitorada pelo watchdog (30 s).
`usb_commands`	4 · 4 KB	Lê bytes da UART0 e entrega comandos (linhas JSON) a uma fila de 8 itens.
Telegram	3 · 10 KB	Consome a fila de avisos e faz o HTTPS. Isola a latência da rede das capturas.

`cam_task` (driver)	núcleo 1 · 4 KB	Recepção DMA dos quadros da câmera.
`sys_evt`, Wi-Fi, lwIP	ESP-IDF	Eventos de rede, pilha TCP/IP e SNTP.

5. Projeto do hardware

O hardware é uma plataforma comercial (COTS). O trabalho de projeto foi identificar a placa, descobrir a pinagem real da câmera, dimensionar a alimentação e definir a montagem mecânica.

5.1 Placa utilizada (conferida em bancada em 02/10/2026)

Item	Observado
Microcontrolador	ESP32-D0WD-V3, revisão 3.0, cristal de 40 MHz (lido com o esptool)
Conversor USB	WCH CH340 (VID:PID 1A86:7523), com circuito automático de reset/boot
PSRAM	8 MB detectados; o ESP32 mapeia 4 MB, que viram heap (~4,3 MB livres no boot)
Flash	16 MB detectados (a página do fabricante informa 4 MB); o firmware usa 4 MB
Câmera	OV2640 (PID 0x26), detectada somente com a pinagem “WROVER KIT” (perfil 2)

5.2 Pinagem da câmera

Apesar do nome comercial ESP32-CAM, a placa da RoboCore não segue a pinagem AI-Thinker, que é a mais comum: o clock da câmera (XCLK) está no GPIO21, e não no GPIO0. Como não havia esquema elétrico, o firmware oferece os dois perfis e a escolha fica salva na NVS, sem recompilar. Na placa, o perfil 1 falhou na sonda SCCB (`ESP_ERR_NOT_FOUND`) e o perfil 2 funcionou.

Sinal	Perfil 1 (AI-Thinker)	Perfil 2 (WROVER KIT) — **usado**
PWDN / RESET	32 / —	— / —
XCLK	0	**21**
SCCB SDA / SCL	26 / 27	26 / 27
D0–D3	5, 18, 19, 21	4, 5, 18, 19
D4–D7	36, 39, 34, 35	36, 39, 34, 35
VSYNC / HREF / PCLK	25 / 23 / 22	25 / 23 / 22

5.3 Alimentação e montagem

- **Fonte:** USB 5 V de pelo menos 2 A, com cabo curto. Wi-Fi e câmera juntos geram picos de corrente; fontes fracas causam reinício por queda de tensão (*brownout*). O motivo de cada reinício é informado na mensagem “Auditor ligado”.
- **Sem eletrônica externa:** o MVP não usa buzzer, LED, RTC nem cartão SD. Isso eliminou o driver do buzzer, o tratamento do botão e o orçamento de componentes da Tarefa 02.
- **Mecânica:** a câmera fica fixa acima do compartimento, enquadrando também parte do entorno, que serve para detectar mudança de cena. O suporte precisa impedir que caixa e câmera se desloquem: um deslocamento de poucos pixels invalida as referências.
- **Iluminação:** ambiente constante. O LED de flash da placa não é usado (decisão do grupo).

- **Rede:** Wi-Fi de 2,4 GHz com WPA2; a placa deve ficar onde o sinal seja bom (RSSI melhor que −75 dBm, informado no status).

6. Projeto do software

6.1 Inicialização

A ordem de boot foi definida pelos problemas encontrados em bancada: NVS, UART e **câmera antes da rede** (a câmera precisa de blocos contíguos de memória interna, que o Wi-Fi fragmenta), depois a tarefa do Telegram, o Wi-Fi e o SNTP. Por fim vêm o evento `boot` (versão, motivo do reset, PSRAM) e o watchdog. Quando a hora chega pelo SNTP, o aparelho envia ao cuidador a mensagem "Auditor ligado", com o motivo do reinício, o que falta configurar e a próxima verificação.

6.2 Máquina de estados do ciclo diário

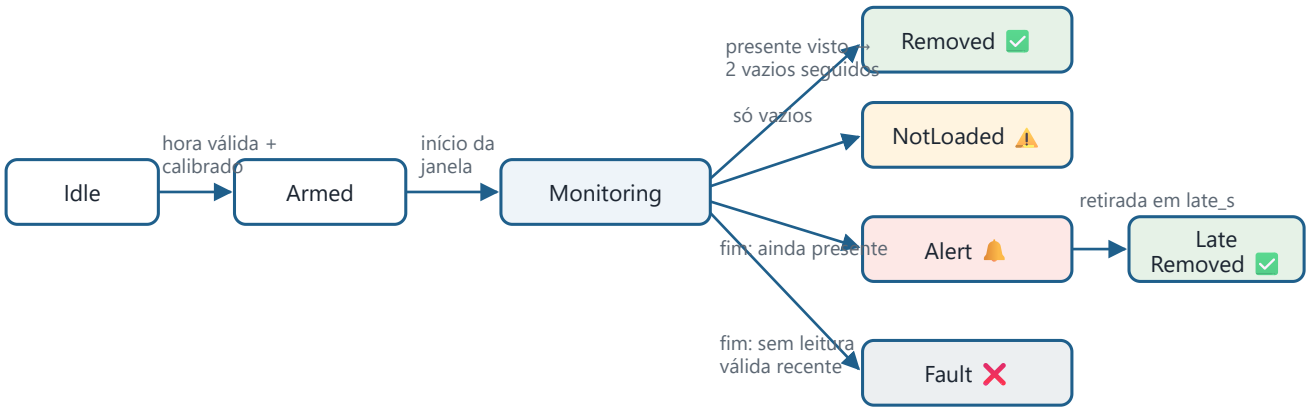


Figura 3 — Máquina de estados `Cycle`. NotLoaded, Alert e Fault enviam foto.

- A presença da dose é verificada **dentro da janela**, não ao armar. Assim o cuidador pode repor a dose a qualquer hora antes do horário.
- **Removed** exige que a dose tenha sido vista e, depois, `confirmations` leituras vazias consecutivas (padrão 2). Se o intervalo entre leituras passar de `interval_s` + 2 s, a sequência recomeça.
- No fim da janela, só uma leitura "presente" recente leva a **Alert**. Qualquer falta de evidência leva a **Fault**: a ausência de dado nunca vira "retirada".
- Em **Alert**, a observação continua por `late_s` (padrão 30 min). Uma retirada nesse período gera **LateRemoved**.
- **Agenda:** `next_window_offset` converte o horário civil na próxima janela, inclusive na virada da meia-noite. Se o aparelho reinicia com a janela de hoje aberta, ela é retomada com o tempo restante. Uma janela concluída nunca se repete no mesmo dia.

6.3 Visão computacional embarcada

1. **Captura:** JPEG QQVGA (160 × 120), qualidade 6. O primeiro quadro é descartado para não reaproveitar imagem antiga. Cada leitura usa dois quadros.
2. **Pré-processamento:** o JPEG é decodificado para RGB888 e convertido para tons de cinza com pesos inteiros.
3. **Características:** a ROI é reduzida por médias de blocos a 32 × 32 = 1024 células. Até 192 pontos do entorno (fora da ROI), cada um a média de 5 × 5 pixels, monitoram mudanças de cena e de iluminação.

4. **Estabilidade:** os dois quadros devem diferir no máximo 8 níveis (média absoluta), na ROI e no entorno. Caso contrário, a leitura é inconclusiva (mão ou movimento na cena).
5. **Referência única:** só o compartimento **vazio** é calibrado (média de 5 quadros). Qualquer objeto que apareça nele, de qualquer formato e em qualquer posição, é tratado como dose: é uma **subtração de fundo**, adequada porque a câmera é fixa.
6. **Compensação de luz:** subtrai-se de todas as células a diferença mediana entre a leitura e o vazio. Uma mudança global de brilho desaparece; uma dose ocupa menos da metade da ROI e não altera a mediana.
7. **Células alteradas:** uma célula mudou se difere do vazio mais que max(20; ruído + 8) níveis. O ruído é o percentil 99 das diferenças medido nos quadros da calibração.
8. **Mancha:** as células alteradas vizinhas são agrupadas (preenchimento de região, conectividade 4). **Presente:** maior mancha ≥ 6 células. **Vazio:** maior mancha ≤ 3 e menos de 6 células alteradas. Entre os dois: inconclusivo.
9. **Rejeição:** imagem escura ou saturada, entorno diferente do vazio (> 20 níveis) ou brilho global alterado além de 20 níveis tornam a leitura inconclusiva, nunca "vazio".

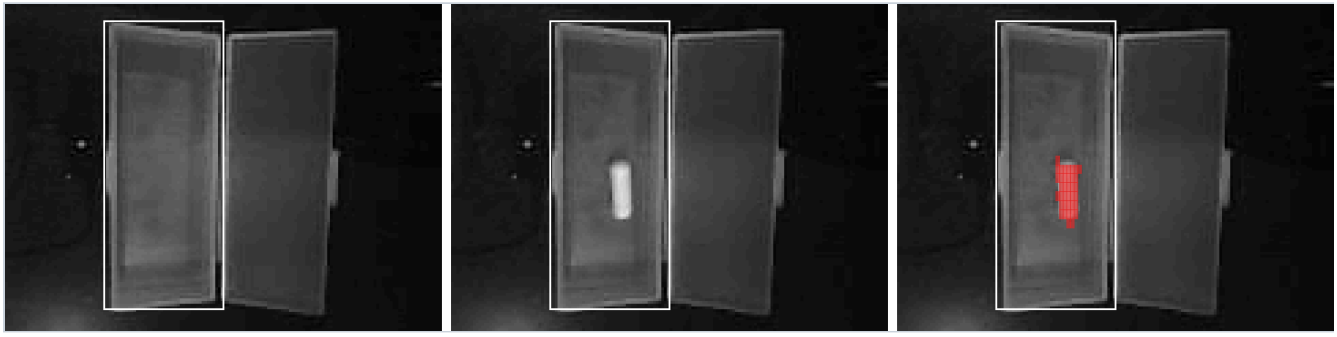


Figura 4 — Imagens reais de 11/10: referência vazia, compartimento com comprimido e células alteradas (vermelho), formando uma mancha de 37 células.

Três ciclos da espiral na visão. (1) A primeira versão comparava com uma foto "cheio" e uma "vazio" pela média das diferenças, e um comprimido pequeno (~4% da ROI) ficava abaixo do ruído do JPEG. (2) A média das 10% maiores diferenças resolveu o comprimido pequeno (31,8 contra 8,9 de ruído). (3) Em 11/10, o teste em bancada com quatro situações mostrou o limite do método: **outra cápsula e o mesmo comprimido deslocado davam inconclusivo**, porque "presente" significava "este comprimido nesta posição". Mas a distância ao vazio separava bem: ~10 no vazio contra 36–46 nos três casos com comprimido. Daí o critério atual, que só pergunta "o compartimento está diferente de vazio?". Nas imagens reais, o vazio gera mancha de 0 células e o comprimido, de 37.

Por que não treinar uma rede neural. Com câmera fixa, "vazio ou ocupado" é resolvido pela subtração de fundo. Uma CNN exigiria centenas de imagens rotuladas, seria mais lenta no ESP32 clássico e mais difícil de explicar. Ela fica como extensão se os ensaios de acurácia mostrarem falhas por iluminação que a compensação não resolva.

Calibração: "Calibrar VAZIO" coleta cinco quadros comparados ao primeiro e é recusada, com mensagem específica, se a cena mudar ou a imagem estiver escura/saturada. "Conferir comprimido" é opcional: informa se uma dose se destaca (tamanho da mancha e diferença), sem salvar nada.

6.4 Notificação ao cuidador

A aplicação nunca espera pela rede: ela coloca o aviso numa fila de 8 itens, que a tarefa do Telegram consome. Cada mensagem tem até 20 tentativas, com espera crescente de até 300 s. Respostas 4xx (token

ou chat errado) não são repetidas, exceto 429 (limite de envio). Sem Wi-Fi, a verificação continua e as mensagens aguardam a reconexão. A foto anexada é o último JPEG analisado, e só é enviada se for da janela atual.

6.5 Persistência e robustez

- **NVS:** um único bloco versionado com configuração, referência vazia e ruído, gravado só ao configurar ou calibrar, nunca a cada quadro (desgaste da flash). As credenciais de Wi-Fi e Telegram ficam em namespaces separados e nunca são ecoadas.
- **Watchdog de tarefas** de 30 s, com reinício em caso de travamento do laço principal.
- **Recuperação:** câmera reinicializada a cada 60 s quando falha (fora da janela); Wi-Fi com reconexão por espera crescente; após reset, a hora volta pelo SNTP e a agenda rearma sozinha.
- **Memória:** buffers RGB (57,6 KB) e cópia do JPEG (16 KB) na PSRAM; mbedTLS aloca na PSRAM. Restam ~95 KB de RAM interna livre com Wi-Fi conectado.

7. Protocolos de comunicação

Enlace	Protocolo	Uso no projeto
Computador ↔ ESP32	USB → UART0 (CH340), 115200 8N1; **JSON Lines** (protocolo próprio, até 511 B por comando)	Configuração, calibração, comandos de teste e eventos de registro. Detalhado abaixo.
ESP32 ↔ câmera	**SCCB** (compatível com I^2C) + **DVP** paralelo de 8 bits com DMA (I2S)	SCCB configura o sensor OV2640 (formato, qualidade JPEG); o DVP transfere os quadros.
ESP32 ↔ roteador	**IEEE 802.11 b/g/n** 2,4 GHz, WPA2-PSK; DHCP	Acesso à Internet em modo estação.
Hora	**SNTP** (UDP 123)	Servidores `a.st1.ntp.br` e `pool.ntp.org`; fuso fixo UTC−3.
ESP32 → Telegram	**HTTPS** (HTTP/1.1 sobre TLS, mbedTLS e pacote de certificados raiz)	Bot API: `sendMessage` (corpo JSON) e `sendPhoto` (`multipart/form-data` com o JPEG).

7.1 Protocolo USB (JSON por linha)

Cada comando é um objeto JSON terminado por LF, com `id` opcional. A placa responde com `event=reply` (repete o `id`, com `ok` e `message`) e emite eventos assíncronos com `ms` (tempo monotônico) e `epoch` (UTC, zero sem hora). Durante uma verificação, só `status` e `cancel` são aceitos. Exemplos:

```
→ {"cmd":"set","hour":8,"minute":0,"window_s":1800,"interval_s":5,"late_s":1800,"id":7}
← {"event":"reply","ms":51234,"epoch":1792152000,"id":7,"ok":true,"message":"..."}
→ {"cmd":"calibrate","label":"empty"}
→ {"cmd":"sample","notify":true}
← {"event":"observation","label":"present","captured":true,"stable":true,
   "distance_present":3.1,"distance_empty":29.4,"distance_context":2.8,"duration_ms":172,...}
← {"event":"transition","from":"monitoring","state":"removed","demo":false,...}
← {"event":"status","state":"armed","wifi":"connected","rssi":-41,"heap":...}
```

Comandos: `status`, `set`, `calibrate`, `clear_calibration`, `snapshot` (imagem em base64), `sample`, `arm` (com `delay_s` para demonstração), `cancel`, `time`, `wifi`, `telegram` e `test_notify`. O painel mascara senha e token no registro, e o firmware nunca os devolve.

7.2 Telegram Bot API

O cuidador cria um bot no BotFather, que fornece o *token*, e envia uma mensagem ao bot. O `chat_id` é obtido em `getUpdates`. Pode ser o chat individual do cuidador ou um grupo, com identificador negativo, para avisar vários familiares. A placa faz requisições POST a `https://api.telegram.org/bot<token>/sendMessage` e `…/sendPhoto`. O certificado do servidor é validado com o pacote de CAs do ESP-IDF.

8. Ferramentas, linguagens e sistema operacional

Categoria	Escolha
Linguagens	C++17 (firmware e núcleo), Python 3 (painel, console e scripts), CMake
Sistema operacional embarcado	**FreeRTOS** (incluso no ESP-IDF): tarefas, filas, semáforos, timers e watchdog
Framework	**ESP-IDF 5.4.1**: esp_wifi, esp_netif, lwIP, esp_netif_sntp, esp_http_client, mbedTLS, NVS, esp_psram
Bibliotecas	`esp32-camera` 2.0.16 (Espressif), cJSON; no PC: pyserial, Tkinter
Compilação e gravação	PlatformIO Core 6.1.18 (plataforma espressif32 6.11.0), esptool; script `tools/build.py` para Windows, macOS e Linux
Testes	Testes nativos do núcleo C++ (311 verificações, `-Wall -Wextra -Werror`), compilados com Zig/clang no Windows, clang no macOS e g++ no Linux; 6 testes Python (unittest)
Integração contínua	GitHub Actions: compilação do firmware e testes no Ubuntu e no macOS a cada envio
Versionamento	Git + GitHub (fork e pull request para integrar ao repositório do grupo)
Sistemas operacionais de desenvolvimento	Windows 11 e macOS
Apoio	VS Code; Telegram (BotFather); análise dos registros JSONL em Python

A IA embarcada da proposta (TinyML/CNN) foi substituída por visão clássica: não havia conjunto de dados rotulado, e a subtração de fundo com uma referência calibrada na própria cena é explicável e verificável em testes sintéticos.

9. Interface com o usuário

9.1 Painel de configuração (computador)

Aplicativo Tkinter (`tools/panel.py`) que detecta a porta da placa automaticamente. Ele mostra a imagem da câmera ampliada, o estado do sistema e o registro de eventos, organizado em três abas:

- **Câmera:** perfil de pinagem, captura de imagem, recorte da ROI com o mouse, “Calibrar VAZIO”, “Conferir comprimido” (opcional) e “Testar leitura”, que classifica a cena e envia o resultado com a foto

ao Telegram.

- **Agenda:** horário, tolerância, intervalo entre leituras, observação após o alerta, ativação da verificação diária, “Teste: verificar em 10 s” e ajuste manual da hora.
- **Wi-Fi e Telegram:** rede, senha, token do bot, `chat_id` e “Enviar mensagem de teste”.

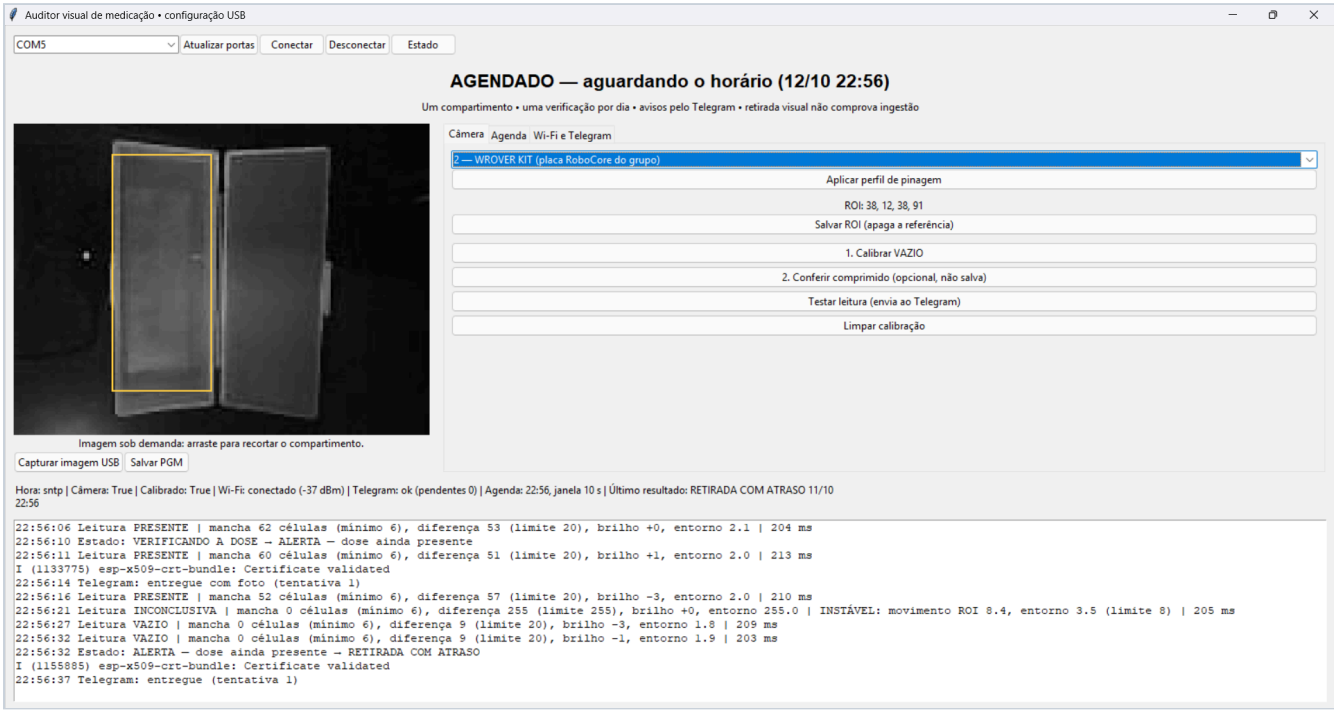


Figura 5 — Painel logo após o ciclo agendado das 22:56 de 11/10: imagem da câmera com a ROI (amarelo), aba Câmera e log com uma leitura a cada ~5 s, o 🔔, a retirada (duas leituras VAZIO) e a confirmação.

9.2 Mensagens ao cuidador (Telegram)

Situação	Mensagem enviada	Foto
Início da janela	⏰ **Hora da dose: 08:00** 📷 Verificando o compartimento por até 30 min.	não
Dose retirada	✅ **Dose tomada** 💊 Dose das 08:00 🕒 Retirada confirmada às 08:03	não
Retirada atrasada	✅ **Dose tomada com atraso** 💊 Dose das 08:00 🕒 Retirada confirmada às 08:41	não
Não retirada	🔔 **Dose NÃO tomada** 💊 Dose das 08:00 ⏰ Ainda no compartimento às 08:30 👀 Continuo observando por mais 30 min e aviso se ela for retirada.	sim
Não reposta	⚠️ **Compartimento vazio no horário** 💊 Dose das 08:00 📭 A dose não foi reposta ou foi retirada antes da hora. Confira a foto.	sim
Falha	❌ **Não foi possível verificar** 💊 Dose das 08:00 📷 Sem imagem confiável (mão na frente, luz ou caixa fora do lugar). Confira a foto.	sim

Ligado / reiniciado	🕯️ **Auditor ligado** 🕒 Relógio: seg 12/10, 07:58 (internet) 🔁 Motivo: energia ligada ✅ Pronto. Próxima verificação: **seg 12/10 às 08:00**	não
Leitura de teste	🔎 **Leitura de teste** · 14:05 💊 **Dose PRESENTE** no compartimento *Mancha de 59 células (mínimo 6) · diferença 45 (limite 20)*	sim

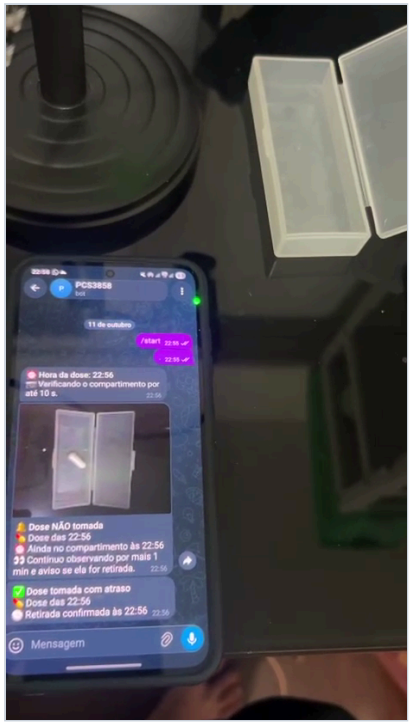


Figura 6 — Quadro do vídeo do ciclo agendado (11/10, 22:56): ⏰ início da janela, 🔔 com a foto do comprimido no compartimento e ✅ após a retirada; ao lado, a caixa já vazia.

10. Protótipo



(a) Montagem (b) ESP32-CAM no suporte (c) Compartimento visto de cima

Figura 7 — Protótipo: (a) a ESP32-CAM presa a um suporte articulado, com a câmera voltada para baixo sobre o porta-comprimidos, ligada pela USB; (b) a placa com o LED de energia aceso; (c) a caixa sob a câmera. Os comprimidos de teste

aparecem em (a).

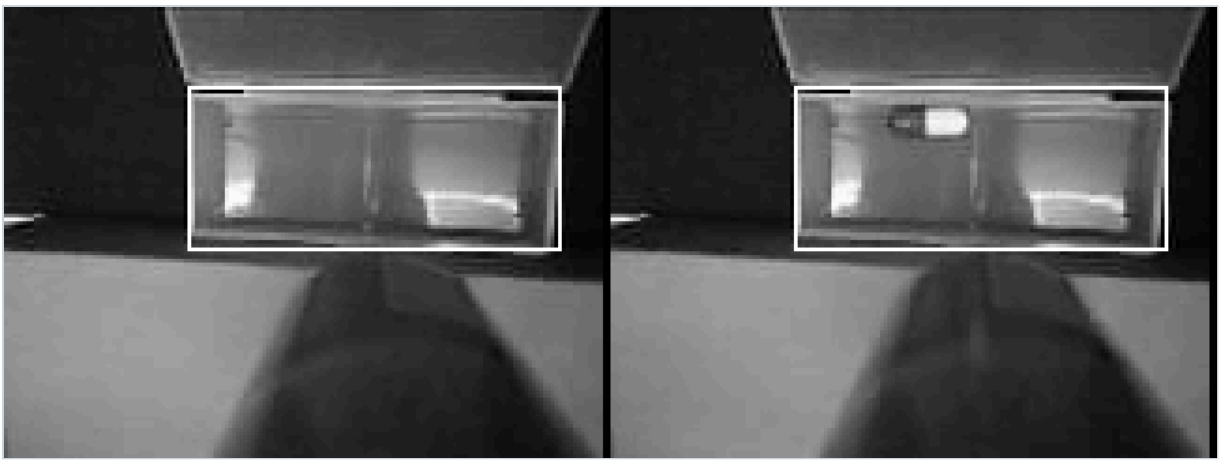


Figura 8 — Imagens da câmera (160 × 120, cinza) de 02/10: compartimento vazio e com um comprimido pequeno, com a ROI marcada.



Figura 9 — Capturas de bancada durante o ajuste do enquadramento: cápsula grande (esquerda) e caixa com a tampa aberta (direita).

Vídeos (anexos em .mp4)

Arquivo	Conteúdo
`video1_ciclo_agendado.mp4`	Verificação disparada pela agenda no horário programado: ⏰ "Hora da dose" no celular; o comprimido continua na caixa e chega 🔔 "Dose NÃO tomada" com foto; o comprimido é retirado e chega ✅ "Dose tomada com atraso" (11/10, 22:56).

11. Verificação e resultados obtidos

11.1 Software

Verificação	Resultado
Núcleo C++ compilado nativamente com `-Wall -Wextra -Werror`	**429 verificações aprovadas**: limites da ROI, referência vazia, comprimido pequeno, diferente, deslocado e duplo, migalha, compensação de luz, ruído, oclusão simulada, persistência temporal, próxima janela (virada de meia-noite, retomada após reinício), not_loaded, alerta com retirada tardia, captura atrasada
Testes Python (protocolo, avaliador, mascaramento de segredos)	6 testes aprovados
Compilação ESP32 (ESP-IDF 5.4.1)	SUCCESS. RAM estática 74.132 B de 327.680 (22,6%); flash 1.191.207 B de 4.063.232 (29,3%)

Integração contínua (GitHub Actions)	Firmware, testes no Ubuntu e testes no macOS aprovados

Os testes sintéticos verificam a lógica, não a acurácia óptica com medicamentos reais.

11.2 Na placa (02/10 a 09/10/2026)

- Gravação pela USB e boot da versão 2.0.0; PSRAM de 8 MB detectada; heap de ~4,3 MB após o boot.
- Câmera OV2640 no perfil 2; leitura completa (2 quadros + análise) em **168–220 ms**, bem abaixo da meta de 2 s (RNF02); 10 de 10 capturas sem aviso do driver.
- Wi-Fi (2,4 GHz, WPA2) conectado com RSSI de −19 a −38 dBm; hora obtida por SNTP.
- **Ciclo agendado completo (11/10, 22:56)**: a agenda disparou no horário programado (⏰ às 22:56:01); o comprimido continuava presente no fim da tolerância (🔔 com foto às 22:56:10); após a retirada, duas leituras vazias confirmaram ✅ "Dose tomada com atraso" às 22:56:32. Antes, uma janela com sombra sobre a caixa (brilho −44) terminou em ❌, como projetado: mudança forte de luz é inconclusiva, nunca "vazio".
- Telegram: mensagem de teste com foto entregue com HTTP 200 na primeira tentativa, cerca de 5 s após o comando; "Testar leitura" entregue com foto e distâncias.
- Calibração com cápsula branca grande aceita; ruído entre quadros parados de ~2,8 (ROI) e ~3,3 (entorno), contra limite 8; 12 de 12 leituras estáveis.
- Métrica top 10% gravada em 11/10 e calibrada com cheio/vazio: o comprimido calibrado foi lido como PRESENTE (diferença 12,7 do cheio e 36,2 do vazio) e o vazio como VAZIO (10,3 do vazio). Outra cápsula e o comprimido deslocado deram INCONCLUSIVA, o que motivou o critério atual (seção 6.3). O novo critério foi gravado e repetido na placa no mesmo dia, com as quatro situações corretas (tabela abaixo).

Situação (11/10, referência só vazia)	Resultado	Maior mancha (mín. 6)	Diferença (limite 20)	Duração
Comprimido usado na calibração anterior	**PRESENTE**	59	45	214 ms
Compartimento vazio	**VAZIO**	0	9	203 ms
Outra cápsula	**PRESENTE**	50	60	222 ms
Mesmo comprimido deslocado	**PRESENTE**	66	47	223 ms

Ruído medido na calibração: 11–12 níveis. A folga é grande (mancha mínima 6 contra 50–66; vazio com 0), mas são quatro leituras: o ensaio de acurácia (RNF03) continua previsto.

12. Problemas enfrentados

Problema	Causa encontrada	Solução
Câmera não inicializava (`ESP_ERR_NOT_FOUND`)	A placa RoboCore usa a pinagem WROVER KIT (XCLK no GPIO21), e não a AI-Thinker	Perfis de pinagem selecionáveis e salvos na NVS; perfil 2 confirmado
Reinício em loop ao salvar o Wi-Fi (`stack overflow in task sys_evt / cam_task`)	Pilhas padrão (2304/2048 B) insuficientes com Wi-Fi e câmera juntos	Pilhas de 4096 B; tarefa da câmera fixada no núcleo 1
Telegram falhava (`mbedtls_ssl_setup -0x7F00`)	O TLS só alocava RAM interna, que estava em ~31 KB livres	`CONFIG_MBEDTLS_DEFAULT_MEM_ALLOC` (usa PSRAM) e buffers de imagem movidos para a PSRAM

Quadros perdidos (`FB-OVF`, `NO-EOI`)	Buffer JPEG automático de 3840 B pequeno para QQVGA	Buffer JPEG fixo de 16 KB
Calibração recusada por "cena instável" com tudo parado	Artefatos do JPEG (qualidade 12) geravam ruído de 4–5 níveis entre quadros; entorno medido em pixels isolados	JPEG qualidade 6 e entorno pela média 5×5: ruído caiu para ~2,8/3,3; mensagens de erro com os valores medidos
Calibração falhava após mexer na câmera	Câmera e caixa sem fixação; ROI grande incluindo partes móveis	Suporte fixo e ROI justa no compartimento
Remédio pequeno não calibrava ("cheio e vazio muito parecidos")	A média da ROI inteira diluía um comprimido que ocupa ~4% da área	Distância pela média das 10% maiores diferenças; margem 8
Outra cápsula ou o comprimido deslocado davam "inconclusivo"	Comparar com uma foto "cheio" faz "presente" significar "este comprimido nesta posição"	Só a referência vazia: compensação de brilho, células alteradas e mancha mínima (subtração de fundo)
Erro "too many initializers" no SNTP	Configuração gerada (`sdkconfig.esp32cam`) desatualizada	Regenerar a configuração sempre que `sdkconfig.defaults` muda (documentado)
Gravação falhava ("porta não existe", "acesso negado")	O painel mantinha a porta serial aberta	Fechar o painel antes de gravar (documentado)
Caminhos longos do ESP-IDF no Windows	Limite MAX_PATH	`build.py` usa `subst` só no Windows
Integrante com macOS	Scripts dependiam do PowerShell e de nomes de porta COM	Ferramentas em Python, detecção do CH340 por VID/PID e CI no macOS

13. Cronograma previsto e realizado

Data	Previsto (Tarefa 02)	Realizado	Situação
04/09	Problema, cenário de uso, recursos	Tema definido e inventário inicial	CONCLUÍDO
11/09	Tema, escopo, metodologia, custos	Proposta preliminar entregue	CONCLUÍDO
18/09	Proposta final e cronograma	Plano de execução (Tarefa 02): requisitos RF/RNF, arquitetura, cronograma, divisão	CONCLUÍDO
25/09	Requisitos, estados, interfaces, critérios de teste	Requisitos e estados definidos; **firmware v1.0.0** (operação pela USB) e testes do núcleo	ADIANTADO
02/10	Arquitetura final, validar componentes	Placa ESP32-CAM conferida; **revisão de escopo**; MVP v2.0.0 com agenda, Wi-Fi, SNTP e Telegram gravado e verificado na placa	ADIANTADO
09/10	Ambiente, firmware inicial, primeira imagem	Calibração em bancada; métrica top 10% para remédio pequeno; ferramentas para macOS; pull request. 11/10: teste com 4 situações e novo critério só com a referência vazia	ADIANTADO
16/10	P1	Relatório, apresentação e demonstração do MVP	EM CURSO
23/10	Agenda e estados; captura estável; classificador preliminar	Agenda e estados já prontos. Antecipar: suporte mecânico fixo e calibração com vários remédios	ANTECIPADO
30/10	Integrar visão, temporização, botão e buzzer	Integração com Telegram já feita; botão e buzzer fora do escopo	REVISTO

06/11	Persistência, registros, falhas; extensões	Persistência e falhas prontas; avaliar extensão a mais compartimentos	REVISTO
13/11 a 27/11	Testes de integração, desempenho, detecção, oclusão, reinício, 20 ciclos	A executar (RNF03, RNF06, RNF07)	PREVISTO
04/12	Retestes, documentação, ensaio da demonstração	A executar	PREVISTO
11/12	P2: protótipo, código, relatório de testes	A executar	PREVISTO

O grupo está **adiantado na implementação**: o fluxo completo, previsto para 30/10, funciona desde 02/10, e o canal remoto, previsto como extensão, já é o modo principal. O ponto que exige atenção é a **validação quantitativa** (acurácia, oclusões e 20 ciclos), que depende do suporte mecânico fixo e da calibração com os remédios definitivos.

14. Divisão de tarefas

Integrante	Responsabilidades (plano da Tarefa 02, ajustadas ao novo escopo)
Gabriel Dias Tambelli	Hardware e integração mecânica: conferência da placa e da câmera, alimentação, suporte e enquadramento, ensaios elétricos (RNF07) e de reprodutibilidade (RNF04); versão inicial do repositório e documentação de implementação.
Henrique Murakami de Paula	Visão computacional e firmware: pipeline de imagem, métrica e limiares, calibração, integração Wi-Fi/SNTP/Telegram, testes na placa, ferramentas multiplataforma; ensaios de detecção (RNF03) e de tempo (RNF02).
Juliana Marcarini Carloni	Firmware e arquitetura: máquina de estados, agenda e persistência em NVS, interface serial e painel, registros, versionamento e documentação do software.

Atividades conjuntas: definição do escopo, testes em bancada, relatório e apresentação.

15. Próximos passos até a P2

1. Concluir o suporte mecânico fixo (câmera e caixa) e documentar distância, ângulo e iluminação (RNF04).
2. Repetir as leituras com comprimidos de cores e tamanhos variados (inclusive brancos sobre fundo claro) e com variação de iluminação ao longo do dia.
3. Ensaio de acurácia: 30 observações por classe fora da calibração e 10 oclusões, com o avaliador de registros (`tools/evaluate.py`) e matriz de confusão (RNF03).
4. 20 ciclos consecutivos e operação de 24 h só na fonte, acompanhando heap mínimo e reinícios (RNF06, RNF07).
5. Medir o tempo entre o fim da janela e a chegada do aviso ao celular.
6. Integrar o pull request ao repositório principal; avaliar a extensão para o organizador semanal (vários compartimentos/ROIs).

16. Referências

- Espressif. *ESP32 Series Datasheet*. Versão incluída em `Documentacao/ESP32_Datasheet.pdf`.
- Espressif. *ESP-IDF Programming Guide v5.4* (Wi-Fi, SNTP/System Time, HTTP Client, NVS, Task Watchdog). docs.espressif.com.

- Espressif. *esp32-camera* v2.0.16. github.com/espressif/esp32-camera.
- Espressif. *CameraWebServer — camera_pins.h* (arduino-esp32 2.0.17), referência dos perfis de pinagem.
- RoboCore. *ESP32-CAM com Gravador Integrado*. robocore.net/wifi/esp32-cam-com-gravador-integrado.
- Telegram. *Telegram Bot API*. core.telegram.org/bots/api.
- NTP.br. *Servidores NTP públicos*. ntp.br.
- FreeRTOS. *FreeRTOS Kernel Reference Manual*. freertos.org.
- Grupo. *Proposta preliminar* e *Tarefa 02 — Consolidação do projeto e plano de execução* (pasta `Documentacao/`).